

HANDS-ON LAB GUIDE

Splunk Fundamentals & Data Onboarding

Day 1 — Hands-On Lab Guide

- Lab 0 — Environment setup
- Lab 1 — Navigate the Splunk interface
- Lab 2 — Upload and index sample logs
- Lab 3 — Identify host, source and source type
- Lab 4 — Basic searches using SPL
- Lab 5 — Filter and format results
- Lab 6 — Field extraction and calculated fields
- Lab 7 — Capstone incident investigation

Vivek Arora

Corporate Trainer, Author & Consultant
StackMaster Tech, Bengaluru

Running scenario: **NovaMart** (fictional e-commerce retailer)
Applies to Splunk Enterprise 9.x / 10.x & Splunk Cloud Platform
Edition 1.0 · September 2026

Contents

Before You Begin	3
Lab Environment Setup	5
Task 0.1 — Start Splunk (choose one option)	5
Task 0.2 — Log in and set your preferences	5
Task 0.3 — Create the novamart index	5
Navigate the Splunk Interface	7
Task 1.1 — Tour Splunk Web	7
Task 1.2 — Explore the Search & Reporting app	7
Task 1.3 — Compare search modes	7
Task 1.4 — Search history and job controls	8
Task 1.5 — Read the Job Inspector	8
Task 1.6 — Events, fields and timestamps	9
Upload and Index Sample Application Logs	10
Part A — Upload the web access logs	10
Part B — Monitor the application log (custom source type)	11
Part C — Upload the payment-gateway JSON	12
Part D — Upload the Linux secure log (with a time zone)	12
Part E — Receive network data over UDP Optional	13
Part F — Universal Forwarder walkthrough Optional / trainer demo	13
Identify Host, Source and Source Type	15
Task 3.1 — Count events by source type and host	15
Task 3.2 — Compare with the source files	15
Task 3.3 — Inspect source and host with metadata	15
Task 3.4 — Check event boundaries	16
Task 3.5 — Check timestamps	16
Task 3.6 — Check the extracted fields	17
Task 3.7 — Look at Splunk's own logs	17
Task 3.8 — Spot the problem (paper exercise)	17
Perform Basic Searches Using SPL	18
Task 4.1 — Terms and field filters	18
Task 4.2 — Boolean operators and precedence	18
Task 4.3 — Wildcards	19
Task 4.4 — Exact phrases and case sensitivity	19
Task 4.5 — Comparisons, IN and field existence	19
Task 4.6 — Time ranges	20
Task 4.7 — Search efficiency	20
Filter and Format Results	22
Task 5.1 — Build a table	22
Task 5.2 — Sort	22
Task 5.3 — Rename for readability	22
Task 5.4 — Remove duplicates	23
Task 5.5 — head, and fields for speed	23
Task 5.6 — Your first transforming commands	23
Task 5.7 — Put it together: a mini report	24

Extract Fields and Create Calculated Fields	25
Part A — Field Extractor (IFX)	25
Part B — rex	25
Part C — eval, if and case	26
Part D — Event types, tags and lookups	28
Capstone: NovaMart Incident Investigation	30
Investigation 3 — Quick hunt Optional	32
Challenge Exercises	33
Appendix A: Answers and Solutions	34
Lab 3	34
Lab 4	34
Lab 5	34
Lab 7 · Investigation 1: Festive Sale outage	34
Lab 7 · Investigation 2: Bastion brute force	36
Lab 7 · Investigation 3: Quick hunt	37
Challenge solutions	38
Appendix B: Lab Troubleshooting	40
Appendix C: Expected Results Card	40

Before You Begin

This lab guide contains the hands-on exercises for **Day 1 — Splunk Fundamentals and Data Onboarding**. Each lab maps to a module in the *Participant Study Guide*. The labs build on each other, so do them in order. Every task shows the **expected result**, so you can check your own work.

Lab map

Lab	Title	Module	Time	You will...
0	Lab environment setup	1	15 min	Start Splunk, set preferences, create the <code>novamart</code> index, unpack the dataset
1	Navigate the Splunk interface	2	25 min	Explore Splunk Web, apps, the Search app, search modes, history and the Job Inspector
2	Upload and index sample application logs	3	45 min	Upload, monitor and receive data with the correct metadata and source types
3	Identify host, source and source type	3	20 min	Validate the onboarded data and troubleshoot
4	Perform basic searches using SPL	4	35 min	Terms, Booleans, wildcards, phrases, time ranges, field filters
5	Filter and format results	4	35 min	<code>fields</code> , <code>table</code> , <code>sort</code> , <code>rename</code> , <code>dedup</code> , <code>head</code> , <code>top / rare</code>
6	Extract fields and create calculated fields	5	50 min	Field Extractor, <code>rex</code> , <code>eval</code> / <code>if</code> / <code>case</code> , event types, tags, lookups
7	Capstone: NovaMart incident investigation	1–5	45 min	Investigate the Festive Sale outage and the bastion brute-force attack
—	Challenge exercises	—	optional	Stretch tasks for fast finishers

What you need

- One of: **Splunk Enterprise** (60-day trial) on Linux, macOS or Windows; **Docker** with the `splunk/splunk` image; or a **Splunk Cloud Platform** trial. Lab 0 covers all three.
- The dataset folder `novamart_dataset` (supplied by your trainer as `novamart_dataset.zip`).
- A modern browser (Chrome, Edge or Firefox), and Python 3 for the optional network-input exercise.
- At least 4 GB of free RAM and 5 GB of free disk space for a local Splunk install.

The dataset folder

NOVAMART_DATASET/

```
|— README.md
|— web/      web01_access.log  web02_access.log      (access_combined, 22,641 events)
|— app/      order_service.log      (novamart:app,      3,042 events)
|— payment/  payment_gateway.json   (novamart:payment,  806 events)
|— auth/     secure.log              (linux_secure,     1,082 events)
|— network/  firewall.log  send_syslog.py        (novamart:firewall,  300 events)
|— lookups/  novamart_products.csv  http_status.csv      threat_intel_ips.csv
|— config/   inputs.conf  outputs.conf  props.conf  transforms.conf  (reference)
```

Conventions

You see	Meaning
Bold text	A button, menu, tab or field in Splunk Web
<i>Settings → Indexes → New Index</i>	A menu path. Click each item in turn.
Dark code block	SPL to type into the search bar, exactly as shown
Green-edged block	Shell commands for a terminal
EXPECTED	What you should see if the step worked
CHECKPOINT	Stop and confirm before moving on
OPTIONAL	Do it if you have time or the right environment

IMPORTANT · TIME RANGE AND TIME ZONE

The NovaMart data is from **14–20 September 2026**. Unless a step says otherwise, set the time range picker to **All time**. Set your user time zone to **(GMT+05:30) Asia/Kolkata** in Lab 0. If you skip that, the counts will still match, but the clock times you see will differ from this guide.

Lab Environment Setup

Goal: a running Splunk instance, your preferences set, an index called `novamart`, and the dataset unpacked.

Task 0.1 — Start Splunk (choose one option)

Option A: Docker (recommended for labs)

TERMINAL — FROM THE FOLDER THAT CONTAINS NOVAMART_DATASET/

```
docker run -d --name splunk \
  -p 8000:8000 -p 8089:8089 -p 5514:5514/udp \
  -e SPLUNK_START_ARGS=--accept-license \
  -e SPLUNK_GENERAL_TERMS=---accept-sgt-current-at-splunk-com \
  -e SPLUNK_PASSWORD='Novamart@2026' \
  -v "$PWD/novamart_dataset:/opt/novamart" \
  splunk/splunk:latest
```

```
docker logs -f splunk      # wait for "Ansible playbook complete", then press Ctrl+C
```

The `-v` option mounts the dataset inside the container at `/opt/novamart`, so Splunk can *monitor* the files in Lab 2. On Windows PowerShell, replace `$PWD` with `${PWD}` and the line-ending `\` with a backtick (```).

Option B: Splunk Enterprise installed locally

LINUX / MACOS (TGZ)

```
sudo tar -xzf splunk- $\langle$ version $\rangle$ -Linux-x86_64.tgz -C /opt
sudo /opt/splunk/bin/splunk start --accept-license      # create admin / Novamart@2026 when prompted
sudo mkdir -p /opt/novamart && sudo cp -r novamart_dataset/* /opt/novamart/
```

On Windows, run the `.msi` installer, accept the licence, create the admin user, and copy the dataset to `C:\novamart\`. Wherever this guide says `/opt/novamart`, use `C:\novamart`.

Option C: Splunk Cloud Platform trial

Log in with the URL and credentials from your trial e-mail. You can do every lab **except** Lab 2 Parts B (monitor) and E (network input); for those, use the *Upload* alternative given in each part.

Task 0.2 — Log in and set your preferences

- 1 Open `http://localhost:8000` and log in as **admin** with your password.
- 2 Click your user name (**Administrator**) in the Splunk bar → **Preferences**.
- 3 On the **Global** tab set **Time zone** = *(GMT+05:30) Asia/Kolkata* and **Default application** = *Search & Reporting*. Click **Apply**.

Task 0.3 — Create the novamart index

- 1 *Settings* → *Indexes* → *New Index*.
- 2 **Index Name:** `novamart` · **Index Data Type:** Events · **App:** Search & Reporting. Leave the other settings at their defaults. Click **Save**.

3

Find `novamart` in the list. **Current Size** is 0 MB and **Event Count** is 0.

On Splunk Cloud Platform: *Settings* → *Indexes* → *New Index*, name `novamart`, then **Save**.

CHECKPOINT

- ☐ You can log in to Splunk Web on port 8000.
- ☐ Your time zone is Asia/Kolkata.
- ☐ The `novamart` index exists and is empty.
- ☐ You know where the dataset folder is (`/opt/novamart` , `C:\novamart` or your laptop, for uploads).

Navigate the Splunk Interface

Goal: find your way around Splunk Web and the Search & Reporting app; compare search modes; use search history, job controls and the Job Inspector.

NovaMart data is not loaded yet, so this lab uses Splunk's own internal logs (`_internal`). Every Splunk instance has them.

Task 1.1 — Tour Splunk Web

- 1 Click the **splunk>** logo to go to the **Home** page. Note the **Apps** panel on the left. Which apps are installed?
- 2 Open each menu in the Splunk bar: **Messages**, **Settings**, **Activity**, **Help**. In **Settings**, find these four items and note which column (Knowledge, Data, System, Users and Authentication) each is in: *Indexes*, *Source types*, *Lookups*, *Licensing*.
- 3 Open *Settings* → *Licensing*. Record the licence type, the daily volume and today's usage.

EXPECTED

Indexes and Source types are under **Data**; Lookups is under **Knowledge**; Licensing is under **System**. A new install shows an *Enterprise Trial* licence of **500 MB** per day.

Task 1.2 — Explore the Search & Reporting app

- 1 Open **Search & Reporting**. Identify the app bar (*Search*, *Analytics*, *Datasets*, *Reports*, *Alerts*, *Dashboards*), the search bar, the time range picker, and the **Search History** panel below the search bar.
- 2 Set the time range picker to **Last 15 minutes** and run:

```
index=_internal
```
- 3 Find each element of the results page and describe what it tells you: the **event count**, the **timeline**, the **fields sidebar** (Selected and Interesting fields), the **Events/Patterns/Statistics/Visualization** tabs, and the **List / Raw / Table** view options.
- 4 Click a bar in the timeline. The list narrows to that interval, and the event count changes, without the search running again. Click **Zoom Out** to go back.
- 5 Expand an event with the **>** icon. Look at its fields and the **Event Actions** menu (Build Event Type, Extract Fields, Show Source).
- 6 In the fields sidebar, click **sourcetype**. The pop-up lists the top values with counts and percentages. Click `splunkd`. Splunk adds `sourcetype=splunkd` to your search and runs it again.

Task 1.3 — Compare search modes

Run this search three times with the time range **Last 60 minutes**, changing the mode selector (below the search bar, on the right) each time: **Fast**, **Smart**, **Verbose**.

```
index=_internal sourcetype=splunkd | top component
```

Mode	Events tab populated?	Fields sidebar populated?	Run time (from Job → Inspect Job)
Fast			
Smart			
Verbose			

Now run a *non-transforming* search (`index=_internal sourcetype=splunkd log_level=WARN`) in **Fast** and then in **Smart** mode, and compare the fields sidebar.

EXPECTED

For the `top` search, only **Verbose** also fills the Events tab; Fast and Smart return only the statistics table, and Verbose is the slowest. For the event search, Fast mode shows only default fields and fields you used in the search. Smart mode discovers fields such as `component` and `log_level`.

Task 1.4 — Search history and job controls

- 1 Go back to the Search app home page (click **Search** in the app bar). Open **Search History**, type `top` in its filter, and re-run your `top component` search with one click.
- 2 Run a deliberately slow search over **All time**, and **Pause** it, **Resume** it, then **Stop** it (buttons to the right of the event count):

```
index=_* | stats count by index, sourcetype
```

- 3 Run it again and click **Job** → **Send to Background**. Open **Activity** → **Jobs** and find your job. Note its **Expires** value, then click **Extend Job Lifetime** (the "clock" action) to 7 days.

Task 1.5 — Read the Job Inspector

Run the following search over **Last 24 hours**, then **Job** → **Inspect Job**:

```
index=_internal sourcetype=splunkd log_level=ERROR
```

Record these values:

Item	Your value
Header message ("...returned X results by scanning Y events in Z seconds")	
<code>scanCount</code>	
<code>eventCount</code>	
<code>runDuration</code>	
Slowest component under <i>Execution costs</i>	

Item	Your value
<code>normalizedSearch</code> (under <i>Search job properties</i>)	

Now remove `log_level=ERROR` and add `ERROR` as a plain term instead (`index=_internal sourcetype=splunkd ERROR`). Compare the scan count and the event count.

DISCUSSION

The plain term `ERROR` matches any event with the word "error" anywhere, including messages such as "0 errors", so its `eventCount` is usually higher. The field filter `log_level=ERROR` is more precise.

Task 1.6 — Events, fields and timestamps

1 Run `index=_internal | head 1` and expand the event. Compare the timestamp shown in the **Time** column with the timestamp text in the raw event.

2 Run the following to see the stored epoch value next to its displayed value:

```
index=_internal | head 1 | eval epoch=_time, shown=strftime(_time, "%Y-%m-%d %H:%M:%S %Z") | table _time, epoch, shown, _raw
```

3 Temporarily change your time zone in **Preferences** to (GMT) UTC and run step 2 again. The `epoch` value stays the same while `_time` and `shown` change. **Change the time zone back to Asia/Kolkata.**

CHECKPOINT

- ☐ You can explain what each part of the Search page does.
- ☐ You can say when to use Fast, Smart and Verbose mode.
- ☐ You have extended a job's lifetime and read `scanCount` / `eventCount` in the Job Inspector.
- ☐ Your time zone is set back to Asia/Kolkata.

Upload and Index Sample Application Logs

Goal: bring all NovaMart sources into the `novamart` index with the correct host, source type and timestamps, using Upload, Monitor and a network input.

Part	File	Method	Source type	Host
A	<code>web/web01_access.log</code> , <code>web02_access.log</code>	Upload	<code>access_combined</code>	web01, web02
B	<code>app/order_service.log</code>	Monitor	<code>novamart:app</code> (new)	app01
C	<code>payment/payment_gateway.json</code>	Upload	<code>novamart:payment</code> (new)	paygw01
D	<code>auth/secure.log</code>	Upload	<code>linux_secure</code> + TZ	bastion01
E	<code>network/firewall.log</code>	UDP 5514 OPTIONAL	<code>novamart:firewall</code> (new)	sender IP
F	Universal Forwarder walkthrough	Demo OPTIONAL	—	—

WATCH OUT · UPLOAD EACH FILE ONLY ONCE

Every upload indexes the file again, and duplicates make all later counts wrong. If you make a mistake, see *Appendix B* → "I uploaded a file twice" to clean the index before re-uploading.

Part A — Upload the web access logs

- 1 **Settings** → **Add Data** → **Upload**. Click **Select File** and choose `web01_access.log`. Click **Next**.
- 2 **Set Source Type**: open the **Source type** drop-down → **Web** → **access_combined**. Check the preview: one line per event, and a `_time` such as `14/09/2026 00:02:...` that matches the `[14/Sep/2026: ...]` text. Click **Next**.
- 3 **Input Settings**: App context = **Search & Reporting**. **Host** → **Constant value** → `web01`. **Index** → `novamart`. Click **Review**, check the summary, then **Submit**.
- 4 Click **Start Searching**. Set the time range to **All time**.
- 5 Repeat steps 1–3 for `web02_access.log` with **Host** = `web02`.

```
index=novamart sourcetype=access_combined | stats count by host
```

EXPECTED

host	count
web01	10856
web02	11785

Part B — Monitor the application log (custom source type)

The application log is still being written in production, so we **monitor** it rather than uploading it once. It also needs a **new source type**, because no pretrained one matches its format:

```
2026-09-18T20:08:08.468+05:30 ERROR [order-service] txn_id=TXN-101536 customer_id=C10038 action=checkout
order_id=ORD-550497 ... status=FAILED error_code=PAY_GW_TIMEOUT response_ms=30464 region=West
```

1 **Settings** → **Add Data** → **Monitor** → **Files & Directories**. **File or Directory**:
`/opt/novamart/app/order_service.log` (Windows: `C:\novamart\app\order_service.log`). Select **Continuously Monitor**. Click **Next**.

2 **Set Source Type**: Splunk guesses a source type. We define our own. Expand **Event Breaks** → **Every Line**. Expand **Timestamp** → **Extraction** = **Advanced...** and enter:

Time Zone	auto (the offset is in the data)
Timestamp format	%Y-%m-%dT%H:%M:%S.%3N%:z
Timestamp prefix	^
Lookahead	30

Click **Apply settings**. The preview now shows one event per line, with `_time` equal to the ISO timestamp.

3 Click **Save As**: Name = `novamart:app`, Description = *NovaMart application services log*, Category = *Application*, App = *Search & Reporting*. Click **Save**, then **Next**.

4 **Input Settings**: Host → *Constant value* → `app01`. Index → `novamart`. **Review** → **Submit** → **Start Searching** (All time).

5 **Watch monitoring in action**. Append a line to the file:

DOCKER

```
docker exec -u splunk splunk bash -c 'echo "$(date -u +%Y-%m-%dT%H:%M:%S).000+00:00 INFO [lab-test] Hello from <your-name>" >> /opt/novamart/app/order_service.log'
```

LINUX / MACOS LOCAL INSTALL

```
echo "$(date -u +%Y-%m-%dT%H:%M:%S).000+00:00 INFO [lab-test] Hello from <your-name>" | sudo tee -a /opt/novamart/app/order_service.log
```

Within a few seconds, search:

```
index=novamart sourcetype=novamart:app lab-test earliest=-15m
```

EXPECTED

`index=novamart sourcetype=novamart:app | stats count` returns **3,043** (3,042 events in the file plus your test line). Your test event appears with the current time. Later expected results in this guide are for subsets of the data, so your test line does not affect them.

Splunk Cloud / cannot monitor? Use **Add Data** → **Upload** with `order_service.log` and the same source-type settings (steps 2–4). Skip step 5; your total will then be 3,042.

LOOK BEHIND THE CURTAIN

Open *Settings* → *Source types*, search for `novamart:app` and click **Edit** → **Advanced**. These are the `props.conf` settings the wizard wrote. Compare them with `config/props.conf` in the dataset.

Part C — Upload the payment-gateway JSON

- 1 *Add Data* → *Upload* → `payment_gateway.json` → **Next**.
- 2 **Source type** → *Structured* → `_json`. Expand **Timestamp** → *Advanced...*: Timestamp format = `%Y-%m-%dT%H:%M:%S.%3N%:z`, Timestamp fields = `timestamp`. **Apply settings**. The preview shows each JSON object as one event with fields such as `gateway` and `latency_ms`.
- 3 **Save As** `novamart:payment` (Category *Structured*, App *Search & Reporting*) → **Next**. Host = `paygw01`, Index = `novamart` → **Review** → **Submit**.

```
index=novamart sourcetype=novamart:payment | stats count by gateway, result
```

EXPECTED (806 EVENTS IN TOTAL)

gateway	result	count
gw-alpha	approved	350
gw-alpha	declined	22
gw-beta	approved	376
gw-beta	declined	23
gw-beta	timeout	35

Nested JSON becomes dotted field names: look for `error.code`, `error.message` and `error.retries` in the fields sidebar.

Part D — Upload the Linux secure log (with a time zone)

Syslog timestamps such as `Sep 17 02:34:51` have **no year and no time zone**. The bastion host runs on IST, so we must tell Splunk. We will add the setting to the `linux_secure` source type *before* uploading.

- 1 *Settings* → *Source types*. Search for `linux_secure` and click it (or **Edit**). Expand **Advanced** → **New setting**: Name = `TZ`, Value = `Asia/Kolkata`. Click **Save**.
- 2 *Add Data* → *Upload* → `secure.log` → **Next**. Source type → *Operating System* → `linux_secure`. In the preview, the first event's raw text starts `Sep 14 00:32:34`. Confirm its `_time` shows `14/09/2026 00:32:34`. If it shows `06:02:34`, the `TZ` setting is not being applied. Click **Next**.
- 3 Host = `bastion01`, Index = `novamart` → **Review** → **Submit**.

```
index=novamart sourcetype=linux_secure "Accepted password"
| eval raw_time=substr(_raw, 1, 15), shown=strftime(_time, "%b %d %H:%M:%S")
| table _time, raw_time, shown, host
```

EXPECTED

1,082 events in total. The single `Accepted password` event has `raw_time` = `shown` = **Sep 17 02:34:51**. If they differ by 5 h 30 m, the `TZ` setting was not applied. Fix it with Appendix B and re-upload.

WHY THIS MATTERS

In the Docker container, Splunk runs on **UTC**. Without `TZ`, every bastion event would be stored 5 h 30 m late, and the 02:11 attack would appear to happen at 07:41 IST. This is Case Study 2 in the Study Guide.

Part E — Receive network data over UDP OPTIONAL

Needs Docker (port 5514/udp published) or a local install. Not available on the Splunk Cloud trial.

- 1 *Settings* → *Add Data* → *Monitor* → *TCP / UDP*. Select **UDP**, Port = `5514`. Click **Next**.
- 2 Source type → **New**: Source Type = `novamart:firewall`, Category = *Network & Security*. Host method = **IP**. Index = `novamart`. **Review** → **Submit**.
- 3 From a terminal on your laptop, replay the firewall events:

```
cd novamart_dataset/network
python3 send_syslog.py 127.0.0.1 5514      # prints: Sent 300 events to udp://127.0.0.1:5514
```

- 4 Search over **Last 15 minutes**:

```
index=novamart sourcetype=novamart:firewall | stats count by action
```

EXPECTED

`allowed` = 241, `blocked` = 59 (300 in total; UDP can drop a few packets on a busy machine). The script re-stamps each event with the current time, which is why they appear in "Last 15 minutes". The `host` is the sender's IP address, e.g. `172.17.0.1` for Docker.

Part F — Universal Forwarder walkthrough OPTIONAL / TRAINER DEMO

In production, NovaMart's servers send logs through Universal Forwarders. If you have a second machine or container, try it:

1. ON THE INDEXER (YOUR SPLUNK INSTANCE): ENABLE RECEIVING

Settings → Forwarding and receiving → Configure receiving → New Receiving Port → 9997 → Save
(Docker: re-create the container with -p 9997:9997 added)

[[2. Start a Universal Forwarder container (host name web03)]]

```
docker run -d --name uf01 --hostname web03 --add-host=host.docker.internal:host-gateway \
  -e SPLUNK_START_ARGS=--accept-license \
  -e SPLUNK_GENERAL_TERMS=--accept-sgt-current-at-splunk-com \
  -e SPLUNK_PASSWORD='Novamart@2026' \
  splunk/universalforwarder:latest
```

[[3. Point the forwarder at the indexer, create a test log, and monitor it]]

```
docker exec -u splunk uf01 /opt/splunkforwarder/bin/splunk add forward-server host.docker.internal:9997 -a
uth admin:Novamart@2026
```

```
docker exec -u splunk uf01 bash -c 'for i in $(seq 1 20); do echo "$(date -u +%FT%T).000+00:00 INFO [web0
3] forwarder test event $i" >> /tmp/web03_test.log; done'
```

```
docker exec -u splunk uf01 /opt/splunkforwarder/bin/splunk add monitor /tmp/web03_test.log -index main -so
urcetype uf_selftest -auth admin:Novamart@2026
```

```
docker exec -u splunk uf01 /opt/splunkforwarder/bin/splunk list forward-server -auth admin:Novamart@2026
```

```
index=main sourcetype=uf_selftest host=web03 | head 10
```

```
index=_internal source=*metrics.log group=tcpin_connections | stats latest(_time) AS last_seen by hostname
```

Expected: `list forward-server` shows `host.docker.internal:9997` under *Active forwards*, and the first search returns the 20 test events with `host=web03`. Compare what the CLI wrote (`docker exec uf01 cat /opt/splunkforwarder/etc/system/local/outputs.conf`) with `config/outputs.conf` in the dataset.

CHECKPOINT · END OF LAB 2

```
| tstats count where index=novamart by sourcetype
```

sourcetype	count
access_combined	22641
linux_secure	1082
novamart:app	3043 (3042 if uploaded)
novamart:firewall	300 (only if Part E done)
novamart:payment	806

Identify Host, Source and Source Type

Goal: prove that every source arrived complete, with the right metadata, event boundaries and timestamps, and practise basic troubleshooting.

Task 3.1 — Count events by source type and host

```
| tstats count where index=novamart by sourcetype, host
```

EXPECTED

sourcetype	host	count
access_combined	web01	10856
access_combined	web02	11785
linux_secure	bastion01	1082
novamart:app	app01	3043*
novamart:firewall	172.17.0.1 (your sender IP)	300 (if Part E done)
novamart:payment	paygw01	806

*3,042 + your test line(s). `tstats` reads only the index files (tsidx), not the raw events, which is why it returns almost instantly. It works only on indexed fields such as `index`, `host`, `source`, `sourcetype` and `_time`.

QUESTION

If the web01 count were 21,712 instead of 10,856, what would most likely have happened? (Answer in Appendix A.)

Task 3.2 — Compare with the source files

Count the lines in each file and compare them with Task 3.1:

```
wc -l novamart_dataset/web/*.log novamart_dataset/app/*.log novamart_dataset/payment/*.json novamart_dataset/auth/*.log
# Windows PowerShell: (Get-Content .\web\web01_access.log).Count
```

EXPECTED

10,856 · 11,785 · 3,042 (+ test lines) · 806 · 1,082. Every line is one event, so the file and the index should match exactly.

Task 3.3 — Inspect source and host with metadata

```
index=novamart | stats count by source, sourcetype, host

| metadata type=hosts index=novamart
| eval firstTime=strftime(firstTime, "%d-%b %H:%M:%S"), lastTime=strftime(lastTime, "%d-%b %H:%M:%S")
| table host, totalCount, firstTime, lastTime
```

1. What is the `source` value of the **uploaded** web logs? And of the **monitored** application log? Why do they look different?
2. Which host has the earliest first event, and which has the latest last event?

EXPECTED

Uploaded files show just the file name (e.g. `web01_access.log`). The monitored file shows its full path (`/opt/novamart/app/order_service.log`). First and last events: web02 14-Sep 00:04:02 → 20-Sep 23:57:00; web01 14-Sep 00:25:56 → 20-Sep 23:55:39; app01 starts 14-Sep 00:00:00 (its last event is your test line); paygw01 14-Sep 05:19:57 → 20-Sep 23:38:16; bastion01 14-Sep 00:32:34 → 20-Sep 22:34:06.

Task 3.4 — Check event boundaries

```
index=novamart | stats count by sourcetype, linecount
```

EXPECTED

Every source type has `linecount=1` only. If you saw `linecount` values of 2 or more for `novamart:app`, several log lines were merged into one event. The fix is *Event Breaks* → *Every Line* (`SHOULD_LINEMERGE=false`).

Task 3.5 — Check timestamps

```
index=novamart sourcetype=novamart:app
| eval raw_ts=substr(_raw, 1, 19), parsed=strftime(_time, "%Y-%m-%dT%H:%M:%S")
| where raw_ts!=parsed AND NOT like(_raw, "%lab-test%")
| stats count
```

This compares the timestamp text at the start of every application-log event with how Splunk parsed it, in your (IST) time zone.

EXPECTED

`count = 0`: every timestamp was parsed exactly. (Your test line is excluded, because it was written in UTC.)

Now check for events in the future, which is a classic sign of time-zone problems:

```
index=novamart earliest=+1m latest=+10y | stats count by sourcetype
```

EXPECTED

No results.

Finally, measure indexing delay (how long after the event happened it was indexed):

```
index=novamart | eval lag_days=round((_indextime - _time)/86400, 1) | stats min(lag_days) max(lag_days) by sourcetype
```

Our historical data shows a lag of several days, because it was generated in September 2026 and indexed today. In production, this value should be seconds; a growing lag signals a forwarding bottleneck.

Task 3.6 — Check the extracted fields

For each source type, run `index=novamart sourcetype=<name>` in **Smart** mode and tick the fields you can see in the sidebar (click **All Fields** if necessary):

Source type	Fields to look for
<code>access_combined</code>	☐ clientip ☐ method ☐ uri_path ☐ status ☐ bytes ☐ useragent ☐ action ☐ product_id
<code>novamart:app</code>	☐ txn_id ☐ customer_id ☐ action ☐ amount ☐ payment_method ☐ status ☐ error_code ☐ region
<code>novamart:payment</code>	☐ gateway ☐ result ☐ latency_ms ☐ card_network ☐ decline_reason ☐ error.code
<code>linux_secure</code>	☐ USER ☐ COMMAND ☐ TTY. Are there fields for the SSH user name or source IP? (No: you will extract them in Lab 6.)

Task 3.7 — Look at Splunk's own logs

```
index=_internal sourcetype=splunkd (component=DateParserVerbose OR component=LineBreakingProcessor) earliest=-4h
| stats count by component, log_level
```

```
index=_internal sourcetype=splunkd component=TailReader OR component=WatchedFile earliest=-4h
| table _time, component, log_level, event_message
```

The second search shows what the monitor input did with `order_service.log`. Look for a message about it being read.

Task 3.8 — Spot the problem (paper exercise)

For each scenario, name the most likely cause and the fix. The answers are in Appendix A.

#	Symptom
1	A new firewall source is sent to <code>index=firewall</code> , but searches return nothing. <code>_internal</code> shows "received event for unconfigured/disabled/deleted index".
2	Java application errors appear as dozens of separate one-line events starting with <code>at com.novamart...</code> .
3	Events from a Windows server in Singapore appear 2 h 30 m in the past compared with the same events on a Bengaluru server.
4	A CSV file dropped daily into a monitored folder is only indexed on the first day.
5	<code> tstats count where index=novamart by sourcetype</code> shows a source type called <code>order_service-too_small</code> .

CHECKPOINT

- ☐ Counts in the index match the source files.
- ☐ All events are single-line and timestamps are correct.
- ☐ You can explain the difference between `source`, `sourcetype` and `host` for each NovaMart source.

Perform Basic Searches Using SPL

Goal: use terms, Boolean operators, wildcards, exact phrases, time ranges and field filters, and check each result against the expected count.

Time range: **All time** unless stated. Write your result in the *Your count* column before you check the *Expected* column.

Task 4.1 — Terms and field filters

#	Search	Your count	Expected
a	<code>index=novamart sourcetype=access_combined</code>		22,641
b	<code>index=novamart sourcetype=access_combined host=web01</code>		10,856
c	<code>index=novamart sourcetype=access_combined status=404</code>		240
d	<code>index=novamart sourcetype=access_combined method=POST</code>		2,871
e	<code>index=novamart sourcetype=novamart:app ERROR</code>		153 (see below)
f	<code>index=novamart sourcetype=novamart:payment result=timeout</code>		35
g	<code>index=novamart sourcetype=linux_secure "Failed password"</code>		466

THINK ABOUT IT · WHY 153 AND NOT 108?

Only 108 events have the log level ERROR. The term `ERROR` also matches the 45 declined-payment WARN events, because they contain `error_code=PAY_DECLINED`. Splunk splits text into terms at "minor breakers" such as `_`, `=`, `-` and `.`, so `error_code` is indexed as `error` and `code`. A bare term matches anywhere in the event. When you need precision, filter on a field instead (you will extract `log_level` in Lab 6).

Task 4.2 — Boolean operators and precedence

#	Search	Your count	Expected
a	<code>index=novamart sourcetype=access_combined status=500 OR status=503</code>		73
b	<code>index=novamart sourcetype=access_combined NOT status=200</code>		385
c	<code>index=novamart sourcetype=access_combined host=web02 status=404 OR status=503</code>		63
d	<code>index=novamart sourcetype=access_combined (host=web02 status=404) OR status=503</code>		83
e	<code>index=novamart sourcetype=linux_secure "Failed password" "invalid user"</code>		250
f	<code>index=novamart sourcetype=linux_secure "Failed password" NOT "invalid user"</code>		216

THINK ABOUT IT

Why do 4.2c and 4.2d return different counts although they contain the same terms? Write out 4.2c with explicit parentheses. *Hint: OR is evaluated before AND.*

TRY THIS MISTAKE ON PURPOSE

Run `index=novamart sourcetype=access_combined status=500 or status=503` (lowercase `or`). It returns **0** events: Splunk now requires `status` to be both 500 and 503 *and* looks for the literal word "or".

Task 4.3 — Wildcards

#	Search	Your count	Expected
a	<code>index=novamart sourcetype=access_combined status=40*</code>		312
b	<code>index=novamart sourcetype=access_combined uri_path=/cart*</code>		2,808
c	<code>index=novamart sourcetype=access_combined uri_path=/cart* action=addtocart</code>		1,404
d	<code>index=novamart sourcetype=novamart:*</code>		3,848 (+ test lines, + 300 firewall)
e	<code>index=novamart sourcetype=access_combined useragent=NovaMartApp*</code>		4,669

Task 4.4 — Exact phrases and case sensitivity

#	Search	Your count	Expected
a	<code>index=novamart sourcetype=novamart:app "timed out"</code>		35
b	<code>index=novamart sourcetype=novamart:app status=failed</code>		35
c	<code>index=novamart sourcetype=novamart:app STATUS=FAILED</code>		0
d	<code>index=novamart sourcetype=linux_secure "accepted password"</code>		1
e	<code>index=novamart sourcetype=linux_secure "Accepted publickey"</code>		76

WHAT YOU JUST PROVED

Field **values** and terms are case-insensitive (4.4b, 4.4d). Field **names** are case-sensitive (4.4c: there is no field called `STATUS`).

Task 4.5 — Comparisons, IN and field existence

#	Search	Your count	Expected
a	<code>index=novamart sourcetype=access_combined status ≥ 500</code>		73
b	<code>index=novamart sourcetype=novamart:app action=checkout response_ms > 5000</code>		40

#	Search	Your count	Expected
c	<code>index=novamart sourcetype=novamart:app payment_method IN (CARD, NETBANKING) status=FAILED</code>		35
d	<code>index=novamart sourcetype=novamart:app error_code=*</code>		80
e	<code>index=novamart sourcetype=novamart:app error_code≠PAY_DECLINED</code>		35
f	<code>index=novamart sourcetype=novamart:app NOT error_code=PAY_DECLINED</code>		2,997 (+ test lines)
g	<code>index=novamart sourcetype=novamart:payment gateway=gw-alpha result=declined</code>		22

THINK ABOUT IT

Explain the big difference between 4.5e and 4.5f.

Task 4.6 — Time ranges

#	Search / time range	Your count	Expected
a	<code>index=novamart sourcetype=access_combined</code> with the picker set to <i>Date Range: Between 18/09/2026 and 18/09/2026</i>		8,657
b	<code>index=novamart sourcetype=access_combined earliest="09/18/2026:19:00:00" latest="09/19/2026:00:00:00"</code>		7,218
c	<code>index=novamart sourcetype=access_combined earliest="09/18/2026:20:00:00" latest="09/18/2026:21:00:00"</code>		1,722
d	<code>index=novamart sourcetype=access_combined status=503</code> with the picker on <i>Last 24 hours</i>		0 (why?)

1. Run 4.6a, then click through the **timeline** to find the busiest hour of 18 September. Which hour is it?
2. In 4.6b, the inline `earliest` / `latest` override the picker. Prove this by setting the picker to *Last 15 minutes* and running it again.

EXPECTED

The busiest hour is **21:00–22:00** on Friday 18 September (1,841 events). Traffic jumps from 154 events at 18:00–19:00 to 1,731 at 19:00–20:00, the moment the Festive Sale opened. 4.6d returns 0 because the data is from September 2026, not from the last 24 hours. Always check the time range first when "there's no data".

Task 4.7 — Search efficiency

Run both searches over **All time**. Open the **Job Inspector** for each and record `scanCount` and the run time.

Search	scanCount	Run time
<code>*503*</code>		
<code>index=novamart sourcetype=access_combined status=503</code>		

EXPECTED

The second search scans far fewer events and is much faster. It also returns exactly the **35** HTTP 503 events. The first returns every event in any index containing "503" anywhere, including byte counts and order IDs. It is also slow because of its leading wildcard.

CHECKPOINT

- ☐ Most of your counts match the expected values.
- ☐ You can explain AND/OR precedence and the difference between `≠` and `NOT`.
- ☐ You can restrict any search to a precise time window.

Filter and Format Results

Goal: turn raw events into clean, readable answers with fields, table, sort, rename, dedup and head, and see the difference between transforming and non-transforming commands.

Time range: All time.

Task 5.1 — Build a table

The web team wants a list of every "page not found" error:

```
index=novamart sourcetype=access_combined status=404
| table _time, host, clientip, method, uri_path, status, useragent
```

EXPECTED

240 rows on the **Statistics** tab, with columns in the order you listed them. The Events tab still exists, but the table is on Statistics.

Task 5.2 — Sort

- 1 Add `| sort - _time`, then `| sort + _time`. Which 404 is the **oldest**, and which is the **newest**?
- 2 Sort by two fields: `| sort host, -_time`. What changes?
- 3 Find the largest single order of the week:

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| table _time, order_id, customer_id, product_id, qty, amount, region
| sort - amount
```

EXPECTED

Newest 404: **20-Sep 23:13:09** from 223.99.81.80 on `/product.screen`. The largest order value is ₹56,997 (qty 3 × NM-1001, the Aurora 5G Smartphone). Seven orders share this value, so their order among themselves may vary.

WATCH OUT

Try `| sort amount` on 812 rows: fine. On more than 10,000 rows, `sort` silently truncates to 10,000 unless you write `sort 0 amount`.

Task 5.3 — Rename for readability

```
index=novamart sourcetype=access_combined status=404
| table _time, host, clientip, uri_path, useragent
| rename host AS "Web Server", clientip AS "Client IP", uri_path AS "Requested Page", useragent AS "Browse
r / Agent"
```

Now add `| sort "Client IP"` at the end. Then try `| sort clientip` instead. What happens, and why?

EXPECTED

`sort clientip` has no effect after the rename, because the field is now called "Client IP". Rename at the end of the search.

Task 5.4 — Remove duplicates

#	Search	Expected rows
a	<code>index=novamart sourcetype=access_combined status=404 dedup clientip table _time, clientip, uri_path</code>	80
b	<code>index=novamart sourcetype=access_combined status=503 dedup host, uri_path table host, uri_path, _time</code>	2
c	<code>index=novamart sourcetype=novamart:app status=FAILED dedup customer_id table customer_id, order_id, amount</code>	35
d	<code>index=novamart sourcetype=access_combined status=404 dedup 2 clientip table clientip, _time</code>	88

QUESTION

In 5.4b, the `_time` shown for web01 is 20:41:12 and for web02 is 20:44:41. Why these times, and not the first 503 on each host?

Task 5.5 — head, and fields for speed

```
index=novamart sourcetype=novamart:app ERROR | head 5
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS | sort - amount | head 3
```

Now compare these two searches in the Job Inspector (look at `runDuration` and the `command.search.kv` cost):

```
index=novamart sourcetype=access_combined | table clientip, status
index=novamart sourcetype=access_combined | fields clientip, status | table clientip, status
```

On this small dataset the difference is small. On production indexes with billions of events, `fields` early in the search saves significant time and memory.

Task 5.6 — Your first transforming commands

#	Search	Expected
a	<code>index=novamart sourcetype=access_combined top limit=5 uri_path</code>	/product.screen 7,918 (34.97%); /category.screen 4,381; / 3,292; /cart.do 2,808; /search 1,281
b	<code>index=novamart sourcetype=access_combined status=404 top limit=3 clientip</code>	192.0.2.66 = 153 (63.75%); 49.90.195.126 = 3; 157.144.83.101 = 2
c	<code>index=novamart sourcetype=access_combined rare limit=3 useragent</code>	Googlebot (39), Nikto scanner (180), ELB-HealthChecker (336)

#	Search	Expected
d	<code>index=novamart sourcetype=access_combined uri_path=/product.screen top limit=5 product_id</code>	NM-1002 914; NM-1001 725; NM-1003 651; NM-6001 504; NM-1005 501
e	<code>index=novamart sourcetype=novamart:app action=checkout top limit=1 customer_id</code>	C10577 with 6 checkouts
f	<code>index=novamart sourcetype=access_combined stats count by host, status</code>	12 rows. web02 has no 401 or 403 events, because only the scanner (which hit web01) produced them.

For each search, note which tab the results appear on, and whether the **Visualization** tab can draw a chart. Then run 5.6a in **Verbose** mode and check the Events tab.

Task 5.7 — Put it together: a mini report

Build a clean table of **failed checkouts** for the payments team, with friendly column names and the most expensive first. Do it yourself before you look at the solution in Appendix A.

Required columns: Time, Order, Customer, Method, Amount (INR), Region. Source: `novamart:app`, `status=FAILED`.

EXPECTED

35 rows. The top two rows are ₹27,999 orders (one ClearView 43in 4K Smart TV each). Every row has Method = CARD or NETBANKING.

CHECKPOINT

- ☐ You can produce a formatted, sorted, renamed table from raw events.
- ☐ You know which event `dedup` keeps.
- ☐ You can tell a transforming command from a non-transforming one by where its results appear.

Extract Fields and Create Calculated Fields

Goal: extract new fields with the Field Extractor and rex, calculate values with eval, if and case, and create your first event type, tag and lookup.

Part A — Field Extractor (IFX)

The payment-service timeout messages contain the gateway name and timeout value as free text, not as key=value pairs:

```
2026-09-18T20:08:08.464+05:30 ERROR [payment-service] txn_id=TXN-101536 Payment gateway gw-beta timed out
after 30000 ms for order ORD-550497 attempt 3 of 3
```

- 1 Search `index=novamart sourcetype=novamart:app "timed out"` (35 events). Expand the first event → **Event Actions** → **Extract Fields**.
- 2 The sample event is pre-selected. Click **Next**. Choose **Regular Expression** → **Next**.
- 3 Highlight `gw-beta` → Field Name `gateway` → **Add Extraction**. Highlight `30000` → `timeout_ms` → **Add Extraction**. Highlight `ORD-550497` → `failed_order` → **Add Extraction**.
- 4 Check the **Preview** tab: the fields should be filled for all matching events. Click **Show Regular Expression** to see what IFX generated. Click **Next**.
- 5 **Validate**: scroll through the matches. If any value is wrong, mark it with **X** (IFX refines the regex). Click **Next**.
- 6 **Save**: Extraction name `nm_gw_timeout`, Permissions = **App**. Click **Finish**.

```
index=novamart sourcetype=novamart:app gateway=* | stats count by gateway, timeout_ms
```

EXPECTED

One row: `gw-beta` | `30000` | **35**. Open **Settings** → **Fields** → **Field extractions** and find `novamart:app` : `EXTRACT-nm_gw_timeout`.

If IFX generated a regex that also matches unrelated lines (e.g. the "Circuit breaker ... gateway gw-beta" warnings), edit it to anchor on `timed out after`, or replace it with: `gateway (?<gateway>\S+) timed out after (?<timeout_ms>\d+) ms for order (?<failed_order>ORD-\d+)`

Part B — rex

B1. Log level and service name

```
index=novamart sourcetype=novamart:app
| rex "^S\s+(?<log_level>[A-Z]+\s+\[(?<service>[^\]]+\])\"
| stats count by service, log_level
```

EXPECTED (11 ROWS, PLUS YOUR LAB-TEST ROW)

auth-service INFO 435 · cart-service INFO 1,404 · inventory-service WARN 60 · order-service ERROR 35 / INFO 807 / WARN 50 · payment-service ERROR 35 / INFO 1 / WARN 9 · platform INFO 168 · search-service ERROR 38

B2. SSH user and source IP

```
index=novamart sourcetype=linux_secure "Failed password"
| rex "Failed password for (invalid user )?(?<user>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"
| stats count by src_ip
| sort - count
| head 3
```

Then find how many different user names the top IP tried:

```
index=novamart sourcetype=linux_secure "Failed password" "203.0.113.77"
| rex "Failed password for (invalid user )?(?<user>\S+) from"
| top limit=20 user
```

EXPECTED

203.0.113.77 = **414**, 10.20.1.15 = 5, then 3 for a NovaMart admin IP. The attacker tried **12** user names: root 96, admin 89, deploy 37, test 35, ubuntu 31, postgres 26, git 22, oracle 18, guest 17, ftpuser 15, jenkins 15, user 13.

B3. Mobile-app platform from the user agent

```
index=novamart sourcetype=access_combined useragent=NovaMartApp*
| rex field=useragent "NovaMartApp/(?<app_version>[\d.]+) \((?<app_os>\w+)"
| stats count by app_os, app_version
```

EXPECTED

Android 4.2.1 = **2,863**; iOS 4.2.1 = **1,806**. Note the `field=useragent` option: rex reads the useragent field instead of `_raw`.

B4. Mask session IDs (mode=sed)

```
index=novamart sourcetype=access_combined JSESSIONID
| rex mode=sed "s/JSESSIONID=[A-Z0-9]+/JSESSIONID=*****/g"
| table _raw | head 5
```

Part C — eval, if and case

C1. Unit conversion and a flag

```
index=novamart sourcetype=novamart:app action=checkout
| eval response_sec = round(response_ms / 1000, 2),
    slow = if(response_ms > 5000, "yes", "no")
| stats count by slow
```

EXPECTED

`no` = 852, `yes` = 40 (the 35 timeouts plus 5 slow-but-successful gw-beta payments).

C2. Classify HTTP status codes with case()

```
index=novamart sourcetype=access_combined
| eval status_class = case(status < 300, "2xx Success",
                           status < 400, "3xx Redirect",
                           status < 500, "4xx Client error",
                           true(),       "5xx Server error")
| stats count by status_class
```

EXPECTED

2xx Success 22,256 · 4xx Client error 312 · 5xx Server error 73 (no 3xx in this data).

C3. Business tiers

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| eval order_tier = case(amount ≥ 10000, "High",
                        amount ≥ 2000,  "Medium",
                        true(),         "Low")
| stats count AS orders, sum(amount) AS revenue by order_tier
| eval revenue = tostring(revenue, "commas")
```

EXPECTED

order_tier	orders	revenue
High	135	3,386,824
Low	343	406,916
Medium	334	1,398,758

Only 17% of orders are "High", but they bring in 65% of revenue.

C4. String concatenation and bytes

```
index=novamart sourcetype=access_combined
| eval request = method . " " . uri_path, kb = round(bytes / 1024, 1)
| top limit=5 request
```

EXPECTED

GET /product.screen 7,918 · GET /category.screen 4,381 · GET / 3,292 · POST /cart.do 1,507 · GET /cart.do 1,301.

C5. Save a calculated field

- 1 *Settings* → *Fields* → *Calculated fields* → *New Calculated Field*. Destination app: *search*. Apply to: **sourcetype** named `novamart:app`. Name: `response_sec`. Eval expression: `round(response_ms/1000, 2)`. **Save**, then set its permissions to **App**.
- 2 Test it with no `eval` in the search: `index=novamart sourcetype=novamart:app status=FAILED | table order_id, response_ms, response_sec`. The `response_sec` values fall between 30.02 and 30.88.

Part D — Event types, tags and lookups

D1. Event type

- 1 Run `index=novamart sourcetype=novamart:app action=checkout status=FAILED`. Click **Save As** → **Event Type**. Name: `nm_checkout_failed`, Tag: `failure`, Color: red. **Save**.

- 2 Test it:

```
eventtype=nm_checkout_failed | stats count by payment_method
tag=failure | stats count
```

EXPECTED

CARD = 29, NETBANKING = 6 (35 in total). `tag=failure` also returns 35.

D2. Tags on field values

- 1 Search `index=novamart host=web01 | head 1`, expand the event, click the arrow next to `host = web01` → **Edit Tags** → Tags: `web_tier` → **Save**. Do the same for `host=web02`.

- 2 Tag `host=bastion01` as `critical_asset`.

```
tag=web_tier status>=500 | stats count by host
tag::host=critical_asset "Accepted" | stats count
```

EXPECTED

web01 = 43, web02 = 30. The bastion search returns 77 accepted logins (76 public-key + 1 password).

D3. Lookups

- 1 *Settings* → *Lookups* → *Lookup table files* → *New Lookup Table File*. Destination app: *search*. Upload `novamart_products.csv`, destination file name `novamart_products.csv`. Repeat for `http_status.csv` and `threat_intel_ips.csv`.
- 2 *Settings* → *Lookups* → *Lookup definitions* → *New Lookup Definition*: Name `novamart_products`, Type *File-based*, Lookup file `novamart_products.csv`. Repeat: `http_status` → `http_status.csv`; `threat_intel_ips` → `threat_intel_ips.csv`.
- 3 For each file and definition, click **Permissions** → *This app only (search)*, Read = Everyone → **Save**.

CHECK

```
| inputlookup novamart_products
```

REVENUE BY CATEGORY

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS
| lookup novamart_products product_id OUTPUT product_name, category
| stats sum(amount) AS revenue by category
| sort - revenue
```

HTTP STATUS DESCRIPTIONS

```
index=novamart sourcetype=access_combined
| lookup http_status status OUTPUT status_description, status_type
| stats count by status_type, status, status_description
```

THREAT INTEL ON WEB CLIENTS (NOTE THE AS MAPPING)

```
index=novamart sourcetype=access_combined
| lookup threat_intel_ips ip AS clientip OUTPUT threat_category, confidence
| search threat_category=*
| stats count by clientip, threat_category, confidence
```

EXPECTED

Revenue: Electronics 4,227,221 · Home & Kitchen 342,070 · Fashion 297,862 · Sports 218,723 · Beauty 67,980 · Books 38,642.

HTTP: Success 22,256; Client Error 312 (404 Not Found 240, 402 Payment Required 45, 403 Forbidden 14, 401 Unauthorized 13); Server Error 73 (500 = 38, 503 = 35).

Threat intel: 198.51.100.23 content_scraper medium 715; 192.0.2.66 web_vuln_scanner high 180. (203.0.113.77 does not appear, because it never touched the web servers.)

D4. Automatic lookup OPTIONAL

Settings → Lookups → Automatic lookups → New: Name `nm_products_auto`, Lookup table `novamart_products`, Apply to **sourcetype** `novamart:app`, Lookup input field `product_id = product_id`, output fields `product_name` and `category`. Save and set permissions to App. Now `index=novamart sourcetype=novamart:app action=checkout | top category` works **without** the `lookup` command.

CHECKPOINT

- ☐ A saved extraction gives you `gateway` and `timeout_ms` automatically.
- ☐ You can write a `rex` with named groups, including an optional group.
- ☐ You can create fields with `eval`, `if` and `case`, and saved `response_sec` as a calculated field.
- ☐ `eventtype=nm_checkout_failed`, `tag=web_tier` and the three lookups all work.

Capstone: NovaMart Incident Investigation

Goal: use everything from Day 1 to investigate two real incidents hidden in the NovaMart data, and write short findings for management.

Work alone or in pairs. For each question, write the SPL you used **and** the answer. Only hints are given here; full solutions are in **Appendix A**. Try not to look until you have an answer.

INVESTIGATION 1 · IT OPERATIONS

"Card payments failing" on Festive Sale night

Brief. On Friday 18 September, soon after 20:00, NovaMart customer support received a burst of complaints: *"I clicked Pay and got an error page."* The Head of Engineering wants a short incident summary by the end of the day: what happened, when, why, who was affected, what it cost, and how it was fixed.

#	Question	Hint
1.1	How many checkout errors did customers see, and between what times (first and last)?	Web logs, 5xx status codes. Use <code>sort</code> and <code>head</code> , or <code>stats min(_time) max(_time)</code> .
1.2	Which page(s) and which web server(s) were affected? Does this point to one bad server or a shared dependency?	<code>stats count by uri_path, host</code>
1.3	What does the application log say was going wrong? Which service logged it?	Search <code>novamart:app</code> for ERROR during the window; extract <code>service</code> with <code>rex</code> , or use your IFX field <code>gateway</code> .
1.4	Which payment methods failed? Why only those? (Prove it from the payment data.)	<code>status=FAILED by payment_method ; then novamart:payment by payment_method, gateway</code> .
1.5	How many customers and how much order value (₹) were affected? Which region was hit hardest?	<code>stats count, dc(customer_id), sum(amount) ; then by region</code> .
1.6	Which products lost the most order value? Use the product names, not the IDs.	<code>lookup novamart_products</code>
1.7	How did gateway latency during 20:00–21:00 compare between the two gateways?	<code>novamart:payment, stats avg(latency_ms) max(latency_ms) count by gateway</code> with a time window.
1.8	Were there early warnings? When and how was the problem fixed? Was there a change ticket?	WARN and INFO events from <code>payment-service</code> in the window.
1.9	Pick one failed order and show its complete story across all three source types in one search.	Search for the order ID across the whole index; <code>table _time, sourcetype, host, _raw</code> .

DELIVERABLE · INCIDENT SUMMARY (5 LINES)

What happened:

When (start-end, duration):

Root cause:

Impact (customers, orders, ₹):

Resolution and follow-up:

INVESTIGATION 2 · SECURITY

Something happened on the bastion at 2 AM

Brief. NovaMart's CISO wants to know whether anyone has been attacking `bastion01`, the SSH jump host into production, and if so, whether they succeeded. There is no SIEM yet; all you have is `linux_secure` in Splunk, and the threat-intelligence lookup.

#	Question	Hint
2.1	How many failed passwords were there in the week, and which source IP is responsible for most of them (count and %)?	<code>rex for src_ip , then top</code>
2.2	What does threat intelligence say about that IP?	<code>lookup threat_intel_ips ip AS src_ip ...</code>
2.3	When did the attack start and stop? How long did it last?	All events containing the IP; <code>stats min(_time) max(_time)</code>
2.4	How many different user names were tried? Which were most targeted? How many attempts used user names that do not exist?	<code>rex for user ; "Invalid user"</code>
2.5	Did the attacker get in? If so, as which user, when, and how?	<code>"Accepted"</code> + the IP
2.6	How does that login differ from the account's normal logins?	<code>rex</code> the auth method and source for all <code>Accepted ...</code> for <code>deploy</code> events
2.7	What did the attacker do after logging in, and when did the session end?	<code>sudo</code> events with <code>TTY=pts/3</code> after the login; extract the full command
2.8	Did the attacker IP appear in any other NovaMart data source?	<code>index=novamart "203.0.113.77" stats count by sourcetype</code> (once you know the IP)

DELIVERABLE · THREE RECOMMENDED ACTIONS FOR THE CISO

Investigation 3 — Quick hunt OPTIONAL

Two other "visitors" left traces in the web logs. Find them using only what you learned today.

1. Which IP produced the most 404 errors, what user agent did it use, and what was it looking for? On which day and during what time window?
2. Which IP made the most requests overall, what user agent did it use, and when does it usually appear? Is it a customer?

Challenge Exercises

For fast finishers. Each challenge uses only Day 1 commands (plus `stats` functions you have seen). Solutions are in Appendix A.

#	Challenge	Task
C1	Conversion funnel	From the application log, count add-to-cart, checkout attempts and successful checkouts for the week. What percentage of add-to-carts became successful orders?
C2	Sale-night leaderboard	Which region generated the most successful revenue between 19:00 and midnight on 18 September? Show all regions with revenue formatted with commas.
C3	Average basket	What was the average successful order value for the week, rounded to 2 decimals?
C4	Why do cards get declined?	From the payment JSON, list decline reasons with their counts.
C5	Card networks	Which card network is used most for CARD payments, and what is its share?
C6	Creature of habit	Show that the scraper visits every day, and at roughly what time. <i>Hint:</i> <code>eval hour=strftime(_time, "%H")</code> .
C7	Admin audit	For each admin who used sudo, list the distinct full commands they ran. <i>Hint:</i> <code>rex</code> both the sudo user and the full command, then <code>dedup</code> .
C8	Privacy by default	Produce a table of failed orders where the customer ID is masked to show only its last two digits (e.g. <code>C105**</code> → <code>C**77</code>). Use <code>eval</code> with <code>substr</code> or <code>replace</code> .
C9	Event type + tag	Create an event type <code>nm_ssh_bruteforce</code> for failed SSH passwords from any source and tag it <code>attack</code> . Then count <code>tag=attack</code> by day.
C10	Field alias thinking	Web events call the client address <code>clientip</code> ; your SSH extraction calls it <code>src_ip</code> . Write one search over both source types that returns a single column <code>src</code> for both. <i>Hint:</i> <code>eval src=coalesce(clientip, src_ip)</code> .

Appendix A: Answers and Solutions

Lab 3

3.1 Question. $21,712 = 2 \times 10,856$: the file was uploaded twice. See Appendix B.

3.8 Spot the problem.

1. The index `firewall` does not exist on the indexer. Create it (on every indexer), then restart the input.
2. Line breaking. Stack-trace lines must be merged with the line that starts with a timestamp: use `LINE_BREAKER` on the timestamp pattern (or `SHOULD_LINEMERGE=true` with `BREAK_ONLY_BEFORE`).
3. Time zone. The Singapore server (UTC+08:00) writes local time without an offset, and Splunk assumes the indexer's zone. Set `TZ = Asia/Singapore` for that host or source type, and re-index.
4. Every CSV starts with the same header line, so the first-256-byte CRC is identical and Splunk thinks it has already read the file. Use `crcSalt = <SOURCE>` (or `initCrcLength`).
5. The file was too small for Splunk to recognise a format, so it created a *too_small* source type. Always assign `sourcetype` explicitly in `inputs.conf`.

Lab 4

4.2 Think about it. 4.2c is read as `host=web02 AND (status=404 OR status=503)` = 48 + 15 = 63. 4.2d is `(host=web02 AND status=404) OR status=503` = 48 + 35 (both hosts) = 83.

4.5 Think about it. `error_code≠PAY_DECLINED` keeps only events that *have* an `error_code` (80) whose value is not `PAY_DECLINED`, i.e. the 35 `PAY_GW_TIMEOUT` events. `NOT error_code=PAY_DECLINED` also keeps the 2,962 events that have no `error_code` at all.

Lab 5

5.4 Question. `dedup` keeps the first event it sees. Results come newest first, so it keeps the *last* 503 on each host.

5.7 Solution.

```
index=novamart sourcetype=novamart:app status=FAILED
| table _time, order_id, customer_id, payment_method, amount, region
| sort - amount
| rename _time AS Time, order_id AS Order, customer_id AS Customer, payment_method AS Method, amount AS "Amount (INR)", region AS Region
```

Lab 7 • Investigation 1: Festive Sale outage

```
1.1
index=novamart sourcetype=access_combined status≥500 earliest="09/18/2026:19:00:00" latest="09/19/2026:00:00:00"
| stats count by status
index=novamart sourcetype=access_combined status=503
| stats count, min(_time) AS first, max(_time) AS last
| eval first=strftime(first, "%H:%M:%S"), last=strftime(last, "%H:%M:%S")
```

35 HTTP 503 errors (plus 1 unrelated 500 from the search service), from **20:07:38** to **20:44:41**, about 37 minutes.

1.2

```
index=novamart sourcetype=access_combined status=503 | stats count by uri_path, host
```

All on `/checkout`: web01 = 20, web02 = 15. Both web servers are affected, so the cause is a **shared dependency**, not one bad server.

1.3

```
index=novamart sourcetype=novamart:app ERROR earliest="09/18/2026:20:00:00" latest="09/18/2026:21:00:00"
| rex "^S\s+(?<log_level>[A-Z]+\s+\[(?<service>[^\]]+\)]\$)"
| search log_level=ERROR
| stats count by service
index=novamart sourcetype=novamart:app "timed out" | stats count by gateway, timeout_ms
```

The term `ERROR` matches 75 events in that hour, including 5 declined-payment WARN events that contain `error_code=`. Filtering on `log_level=ERROR` leaves 70 ERROR events: `payment-service` 35 ("Payment gateway gw-beta timed out after 30000 ms ... attempt 3 of 3") and `order-service` 35 (`status=FAILED error_code=PAY_GW_TIMEOUT`). All 35 timeouts were on **gw-beta**.

1.4

```
index=novamart sourcetype=novamart:app status=FAILED | stats count by payment_method
index=novamart sourcetype=novamart:payment | stats count by payment_method, gateway
```

CARD = 29, NETBANKING = 6. The payment data shows that CARD and NETBANKING are **always routed to gw-beta** (336 and 98 events), while UPI and WALLET go to gw-alpha. So only gw-beta's customers were hit. (COD orders don't use a gateway.)

1.5

```
index=novamart sourcetype=novamart:app status=FAILED
| stats count AS failed_orders, dc(customer_id) AS customers, sum(amount) AS value
index=novamart sourcetype=novamart:app status=FAILED | stats count, sum(amount) AS value by region | sort
- count
```

35 orders, 35 customers, ₹2,01,760 (201,760). By region: North 13, South 9, West 8, East 5.

1.6

```
index=novamart sourcetype=novamart:app status=FAILED
| lookup novamart_products product_id OUTPUT product_name
| stats count AS orders, sum(amount) AS value by product_name
| sort - value | head 4
```

Aurora 5G Smartphone ₹56,997 (3 orders) · ClearView 43in 4K Smart TV ₹55,998 (2) · Digital Air Fryer ₹29,995 (3) · PulseBeat Wireless Earbuds ₹14,994 (6, the most orders).

1.7

```
index=novamart sourcetype=novamart:payment earliest="09/18/2026:20:00:00" latest="09/18/2026:21:00:00"
| stats count, avg(latency_ms) AS avg_ms, max(latency_ms) AS max_ms by gateway
| eval avg_ms=round(avg_ms, 0)
```

gw-alpha: 41 payments, average $\approx$ 470 ms. gw-beta: 62 payments (35 timeouts, 27 approved, 5 of them only after 9–24 s), average $\approx$ 18,501 ms, max 30,883 ms. For comparison, gw-beta's normal average (excluding timeouts, whole week) is $\approx$ 607 ms.

1.8

```
index=novamart sourcetype=novamart:app "payment-service" (WARN OR INFO) earliest="09/18/2026:20:00:00" latest="09/18/2026:21:00:00"
| table _time, _raw
```

Nine `Circuit breaker HALF_OPEN for gateway gw-beta` warnings every 5 minutes from **20:05:01** to **20:45:01** (failure rate 78–93%). These were early warnings that nobody alerted on. The fix: *"Failover complete: CARD and NETBANKING traffic rerouted from gw-beta to gw-alpha (change_id=CHG-20918)"* at **20:45:20**. There were no further 503s after it.

1.9

```
index=novamart ORD-550497 | table _time, sourcetype, host, _raw | sort _time
```

Four events: web02 POST /checkout → 503 at 20:07:38; the payment JSON `result=timeout` and `latency_ms=30464`; payment-service ERROR (gw-beta timed out); order-service ERROR (FAILED, ₹18,999, CARD, region West).

MODEL INCIDENT SUMMARY

What: 35 customers received HTTP 503 errors at checkout during the Festive Sale. **When:** 18-Sep 20:07:38 – 20:44:41 IST (~37 min). **Root cause:** payment gateway gw-beta, which handles all CARD and NETBANKING payments, stopped responding; every attempt timed out after 30 s × 3 retries. **Impact:** 35 failed orders worth ₹2,01,760, 35 customers (North region hardest hit); 5 other gw-beta payments succeeded but took 9–24 s. **Resolution:** traffic failed over to gw-alpha at 20:45:20 (CHG-20918). **Follow-up:** alert on circuit-breaker warnings (first seen 20:05:01, 2.5 minutes before the first customer error) and on checkout 5xx; contact the 35 affected customers.

Lab 7 • Investigation 2: Bastion brute force

2.1 / 2.2

```
index=novamart sourcetype=linux_secure "Failed password"
| rex "from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"
| top limit=3 src_ip
| lookup threat_intel_ips ip AS src_ip OUTPUT threat_category, confidence, first_seen
```

466 failed passwords; **203.0.113.77 = 414 (88.8%)**. Threat intel: `ssh_bruteforce`, confidence `high`, first seen 2026-08-30.

2.3

```
index=novamart sourcetype=linux_secure "203.0.113.77"
| stats count, min(_time) AS start, max(_time) AS end
| eval duration_min=round((end-start)/60, 1), start=strftime(start, "%d-%b %H:%M:%S"), end=strftime(end, "%d-%b %H:%M:%S")
```

665 events, from **17-Sep 02:11:08** to **02:34:51** (the last failure was at 02:34:44), about 23.7 minutes, with a guess every 2–5 seconds.

2.4

```
index=novamart sourcetype=linux_secure "Failed password" "203.0.113.77"
| rex "Failed password for (invalid user )?(?<user>\$+) from"
| top limit=12 user
index=novamart sourcetype=linux_secure "Invalid user" "203.0.113.77" | stats count
```

12 user names: root 96, admin 89, deploy 37, test 35, ubuntu 31, postgres 26, git 22, oracle 18, guest 17, ftpuser 15, jenkins 15, user 13. 250 attempts used user names that don't exist on the host.

2.5 / 2.6

```
index=novamart sourcetype=linux_secure "Accepted" "for deploy"
| rex "Accepted (?<auth_method>\$+) for (?<user>\$+) from (?<src_ip>\$+)"
| stats count, min(_time) AS first, max(_time) AS last by user, auth_method, src_ip
| eval first=strftime(first, "%d-%b %H:%M"), last=strftime(last, "%d-%b %H:%M")
```

Yes. Accepted password for deploy from 203.0.113.77 port 51522 at 17-Sep 02:34:51. Normally deploy logs in with a **publickey** from the internal address 10.20.1.15 during working hours (23 times this week). This was the only password login, from an external IP, at 2:34 AM.

2.7

```
index=novamart sourcetype=linux_secure earliest="09/17/2026:02:34:00" latest="09/17/2026:03:00:00" (sudo 0
R "session")
| rex "sudo:\$+(?<sudo_user>\$+) :.*COMMAND=(?<full_command>.+)"
| table _time, sudo_user, full_command, _raw
```

02:35:29 /usr/bin/id · 02:35:52 /bin/cat /etc/shadow · 02:36:26 /usr/sbin/useradd -m -s /bin/bash support2 · 02:37:01 /usr/bin/passwd support2 · 02:37:42 /usr/bin/crontab -l. Session closed at 02:41:41. The attacker read the password hashes and created a **backdoor account**.

2.8

```
index=novamart "203.0.113.77" | stats count by sourcetype
```

Only linux_secure (665). The IP did not touch the web tier.

MODEL RECOMMENDATIONS

1. Contain: disable the support2 account, rotate the deploy credentials and all passwords whose hashes were in /etc/shadow, check cron and SSH keys for persistence, and block 203.0.113.77.
2. Harden: disable SSH password authentication (PasswordAuthentication no), allow SSH only from the corporate VPN, add fail2ban or equivalent rate limiting.
3. Detect: save an event type and an alert for more than 20 failed passwords from one IP in 5 minutes, and for any successful password login to bastion01 from a non-internal IP (Day 2).

Lab 7 • Investigation 3: Quick hunt

1. 192.0.2.66: 153 of its 180 requests returned 404 (plus 14 × 403 and 13 × 401). User agent Nikto/2.5.0, a vulnerability scanner. It probed /wp-login.php, /.env, /.git/config, /etc/passwd, /backup.zip, /phpmyadmin/ and similar paths on web01, on 16-Sep between 11:02:08 and 11:08:05.

2. 198.51.100.23: 715 requests, python-requests/2.31.0, on web02 every day at about 03:00–03:40 (around 100 requests per day), scraping product pages. It is a content scraper, not a customer. (10.20.0.5, with 336 requests, is the load balancer's health check.)

Challenge solutions

C1 FUNNEL

```
index=novamart sourcetype=novamart:app action IN (addtocart, checkout)
| eval stage=case(action="addtocart", "1 add-to-cart", action="checkout" AND status="SUCCESS", "3 check
out success", action="checkout", "2 checkout attempt (failed/declined)")
| stats count by stage
```

1,404 add-to-carts → 892 checkout attempts (812 success + 45 declined + 35 failed) → 812 successful.
812 / 1,404 = **57.8%**.

C2 SALE-NIGHT LEADERBOARD

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS earliest="09/18/2026:19:00:00" late
st="09/19/2026:00:00:00"
| stats sum(amount) AS revenue by region | sort - revenue | eval revenue=tostring(revenue, "commas")
```

South 1,015,004 · North 985,722 · West 571,358 · East 346,337 (382 orders, ₹29,18,421 in five hours).

C3 / C4 / C5

```
index=novamart sourcetype=novamart:app action=checkout status=SUCCESS | stats avg(amount) AS aov | eval ao
v=round(aov, 2)
index=novamart sourcetype=novamart:payment result=declined | top decline_reason
index=novamart sourcetype=novamart:payment payment_method=CARD | top card_network
```

C3: ₹6,394.70. C4: do_not_honor 12, otp_failed 12, insufficient_funds 11, expired_card 10. C5: VISA 153 (45.5%), MASTERCARD 110, RUPAY 73.

C6 SCRAPER

```
index=novamart sourcetype=access_combined clientip=198.51.100.23
| eval day=strftime(_time, "%a %d"), hour=strftime(_time, "%H")
| stats count by day, hour
```

Seven rows, one per day, all in hour **03**, with 97–109 requests each.

C7 ADMIN AUDIT

```
index=novamart sourcetype=linux_secure sudo
| rex "sudo:\s+(?<sudo_user>\S+) :.*COMMAND=(?<full_command>.+)$"
| dedup sudo_user, full_command | table sudo_user, full_command | sort sudo_user
```

C8 MASKING

```
index=novamart sourcetype=novamart:app status=FAILED
| eval customer_masked = "C***" . substr(customer_id, -2)
| table _time, order_id, customer_masked, amount
```

C9 EVENT TYPE + TAG

```
(event type search) index=novamart sourcetype=linux_secure "Failed password"
tag=attack | eval day=strftime(_time, "%a %d-%b") | stats count by day
```

Thursday 17-Sep stands out with the 414 brute-force attempts; the other days show only a handful of background failures.

C10 ONE COLUMN FOR TWO FIELD NAMES

```
index=novamart (sourcetype=access_combined status=404) OR (sourcetype=linux_secure "Failed password")
| rex "from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3}) port"
| eval src=coalesce(clientip, src_ip)
| top limit=5 src
```

Appendix B: Lab Troubleshooting

Problem	Fix
"No results found"	Set the time range to All time . Check <code>index=novamart</code> (not <code>main</code>). Check the spelling of the source type: <code>novamart:app</code> , not <code>novamart_app</code> .
I uploaded a file twice	Give yourself the <code>can_delete</code> role (<i>Settings</i> → <i>Users</i> → <i>admin</i> → <i>Assign roles</i> → <i>can_delete</i>). Then run, for example, <code>index=novamart source=web01_access.log delete</code> and upload the file once more. (<code>delete</code> hides events from search; it does not free disk space.) Alternatively, stop Splunk and run <code>splunk clean eventdata -index novamart -f</code> to empty the whole index.
Bastion events are 5 h 30 m off	<code>TZ</code> was not set before upload. Add <code>TZ = Asia/Kolkata</code> to <code>linux_secure</code> (Lab 2 Part D step 1), delete the <code>secure.log</code> events as above, and upload again.
Application-log events contain several lines	Edit source type <code>novamart:app</code> → Event Breaks = Every Line (<code>SHOULD_LINEMERGE=false</code>). Delete and re-index.
The monitor input doesn't pick up the file (Docker)	Check the volume mount: <code>docker exec splunk ls -l /opt/novamart/app</code> . The file must be readable by the <code>splunk</code> user.
Payment events have the wrong time	In the <code>_json</code> source type, set Timestamp fields = <code>timestamp</code> and the format <code>%Y-%m-%dT%H:%M:%S.%3N%:z</code> .
Fields such as <code>clientip</code> are missing from the sidebar	You are in Fast mode. Switch to Smart or Verbose.
UDP events don't arrive	Was the container started with <code>-p 5514:5514/udp</code> ? Is the input enabled (<i>Settings</i> → <i>Data inputs</i> → <i>UDP</i>)? Is a local firewall blocking UDP 5514?
The lookup fails with "Could not find all of the specified lookup fields"	Check the lookup column names with <code> inputlookup novamart_products</code> , and the permissions on both the lookup file and the definition.
The IFX field also matches the wrong events	Edit the regex so that it includes the literal text <code>timed out after</code> (see Lab 6 Part A).

Appendix C: Expected Results Card

Check	Value	Check	Value
web01 / web02 events	10,856 / 11,785	gw-beta timeouts	35
novamart:app events	3,042 (+ test lines)	Failed order value	₹2,01,760
novamart:payment events	806	Successful orders / revenue	812 / ₹51,92,498
linux_secure events	1,082	Failed SSH passwords	466 (414 from 203.0.113.77)
novamart:firewall events	300 (optional)	Attacker login	deploy, 17-Sep 02:34:51
HTTP 404 / 5xx	240 / 73	Backdoor user	support2
HTTP 503 (outage)	35, 20:07:38–20:44:41	Scanner / scraper	192.0.2.66 / 198.51.100.23
Web events on 18-Sep	8,657	Top category by revenue	Electronics ₹42,27,221